Seminar 06 — Routing & App Architecture

PB138 — Basics of Web Development

"A URL is a user interface — treat it like one"

Agenda

1. **Why routing?** — URLs, navigation, the problem with a single component (~5 min)
2. **TanStack Router** — route trees, file-based routing, params, search params (~30 min)
3. **Building the app** — layouts, nested routes, guards, loaders, code splitting (~25 min)
4. **TanStack Start** — what it adds on top of Router (~10 min)

Why Routing?

Your app has multiple pages — your URL should reflect that

The Problem

Right now, our app looks like this:

```
// App.tsx — everything in one component
function App() {
  return (
    <div>
      <StudentsPage />
      <CoursesPage />
    </div>
  );
}
```

Problems:

- **No URLs** — you can't link to `/students` or `/courses`
- **No navigation** — everything renders at once
- **No browser history** — back/forward buttons don't work
- **No sharing** — you can't send someone a link to a specific page

What is Client-Side Routing?

Maps URLs to components — without a full page reload.

```
/students        →  <StudentsPage />
/courses         →  <CoursesPage />
/courses/abc123 →  <CourseDetailPage />
```

The browser URL changes, the right component renders — but it's still a **single-page app** (SPA). No server round-trip for each page.

Anatomy of a URL

```
https://example.com /courses/123/enrollments ?semester=spring&page=2
                    ──────────────────────── ──────────────────────
                              path                  search params
                         (route + params)          (query string)
```

Route path — identifies which page: `/courses`, `/courses/123/enrollments`

Path parameters — dynamic segments: `/courses/$id/enrollments` → `/courses/123/enrollments`

Used for **identity** — which specific resource are we looking at?

Search parameters — key-value pairs after `?`: `?semester=spring&page=2`

Used for **filtering, sorting, pagination** — how do we want to view the data?

```
/courses/123/enrollments?semester=spring&page=2
         ───              ──────────────────────
       "which"            "how to display"
```

Why TanStack Router?

Feature	React Router	TanStack Router
Type-safe params	❌ strings	✅ fully typed
Type-safe search params	❌ manual	✅ with validation
Built-in data loading	partial	✅ loaders
File-based routing	❌	✅ via plugin
Devtools	❌	✅

Designed for TypeScript from the ground up. Typos are caught at compile time, not in production.

The Real Problem with Untyped Routing

With React Router, routes are **strings**. Typos hide until a user hits a broken link.

```
// React Router — string concatenation for routes
<Link to={"/cousres/" + course.id}>Detail</Link>;
//         ^^^^^^^^ typo — blank page, no error, good luck

// React Router — search params are raw strings, parse manually
const [searchParams] = useSearchParams();
const page = Number(searchParams.get("page")) || 1;
const semester = searchParams.get("semester") ?? "spring";
// No validation, no types — get("semster") returns null, no warning
```

```
// TanStack Router — typed routes, validated search params
<Link to="/courses/$id" params={{ id: course.id }}>
  Detail
</Link>;
// ✅ typo in route path → TS error, autocomplete for routes + params

const { page, semester } = Route.useSearch();
// ✅ Zod validates & parses — page is number, semester is enum
// ✅ typo in field name → TS error
```

TanStack Router — Core Concepts

Type-safe routing for React

Route Tree

TanStack Router uses a **route tree** — a hierarchy of routes that mirrors your URL structure.

```
__root.tsx                → Root Layout
├── index.tsx             → /
├── students.tsx          → /students
├── courses/
│   ├── route.tsx         → /courses (layout)
│   ├── index.tsx         → /courses (page)
│   └── $id.tsx           → /courses/$id
└── about.tsx             → /about
```

Each route is a **node** in the tree. Child routes inherit their parent's layout.

Two ways to define routes: **code-based** (manual) or **file-based** (generated).

Code-Based Routes

```
import { createRootRoute, createRoute, createRouter } from "@tanstack/react-router";

const rootRoute = createRootRoute({
  component: RootLayout,
});

const indexRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/",
  component: HomePage,
});

const studentsRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/students",
  component: StudentsPage,
});

const routeTree = rootRoute.addChildren([indexRoute, studentsRoute]);
const router = createRouter({ routeTree });
```

Works — but gets verbose fast. Let's look at file-based routing.

File-Based Routing

TanStack Router has a **Vite plugin** (`@tanstack/router-plugin`) that generates the route tree from your file system.

```
src/routes/
├── __root.tsx              → Root layout (navbar, footer)
├── index.tsx               → /
├── about.tsx               → /about
├── students.tsx            → /students (simple route — no children)
├── courses/
│   ├── route.tsx           → /courses layout (shared UI + <Outlet />)
│   ├── index.tsx           → /courses (list page)
│   └── $id.tsx             → /courses/$id (detail page)
```

`$` → dynamic param. `__root` → root layout. `route.tsx` in a folder → layout wrapping children.

Setting Up File-Based Routing

```
bun add @tanstack/react-router @tanstack/router-plugin
```

```
// vite.config.ts
import { tanstackRouter } from "@tanstack/router-plugin/vite";

export default defineConfig({
  plugins: [tanstackRouter(), react()],
});
```

The plugin watches `src/routes/` and auto-generates `routeTree.gen.ts`. Create a new file → route is immediately available. No manual wiring, no restart.

Route File Convention

Each route file exports a `Route` using `createFileRoute`:

```tsx
// src/routes/students/index.tsx
import { createFileRoute } from "@tanstack/react-router";

export const Route = createFileRoute("/students/")({
  component: StudentsPage,
});

function StudentsPage() {
  return <h1>Students</h1>;
}
```

The path string (`"/students/"`) is **auto-generated** by the plugin — it must match the file location. The tooling enforces this.

The Root Layout

The root route wraps **every page** — navbar, footer, providers go here.

```tsx
// src/routes/__root.tsx
import { Outlet, Link } from "@tanstack/react-router";

function RootLayout() {
  return (
    <div className="min-h-screen">
      <nav className="border-b p-4">
        <Link to="/">Home</Link>
        <Link to="/students">Students</Link>
        <Link to="/courses">Courses</Link>
      </nav>

      <main className="p-6">
        <Outlet /> {/* ← child route renders here */}
      </main>
    </div>
  );
}
```


`<Outlet />` is the slot where the matched child route renders.

Navigation — Link Component

```
import { Link } from "@tanstack/react-router";

// Basic link
<Link to="/students">Students</Link>

// With path params — fully typed!
<Link to="/courses/$id" params={{ id: course.id }}>
  {course.name}
</Link>

// With search params
<Link to="/courses" search={{ semester: "spring" }}>
  Spring Courses
</Link>

// Active styling — automatic
<Link
  to="/students"
  activeProps={{ className: "font-bold text-blue-600" }}
>
  Students
</Link>
```

`Link` validates `to`, `params`, and `search` at compile time. Wrong route name? TypeScript error.

Programmatic Navigation

```
// src/routes/students/index.tsx
export const Route = createFileRoute("/students/")({
  component: StudentsPage,
});

function StudentsPage() {
  const navigate = Route.useNavigate();

  const onSubmit = async (data: NewStudent) => {
    const student = await createStudent(data);

    // Type-safe — knows which routes exist
    navigate({ to: "/students/$id", params: { id: student.id } });
  };

  return <form onSubmit={handleSubmit(onSubmit)}>...</form>;
}
```

Before You Start

```
# 1. Clone the repo & install dependencies
bun i

# 2. Start PostgreSQL
docker compose up -d

# 3. Copy environment variables
cp apps/server/.env.example apps/server/.env

# 4. Apply migrations & seed the database
bun run db:push
bun run db:seed

# 5. Start the server + frontend
bun run dev
```

Open http://localhost:5173 — you should see the app shell. API docs at http://localhost:3000/api-docs.

Task 1 — Create Routes & Navigation (~10 min)

The skeleton has TanStack Start set up with `__root.tsx` ready. Your job:

1. **Create two routes:** `routes/students/index.tsx` and `routes/courses/index.tsx`
    - Move the existing `StudentsPage` and `CoursesPage` into their route files
2. **Add navigation** in `__root.tsx`:
    - Add `<Link>` components to `/students` and `/courses`
    - Use `activeProps` to highlight the current page
3. Add a default route (`/`) that redirects to `/students`

Verify: Click links → URL changes, correct page renders, back button works.

Path Parameters

Dynamic segments in URLs — e.g. `/courses/123`.

```
// Route definition — $ marks the dynamic segment
const courseDetailRoute = createRoute({
  getParentRoute: () => coursesRoute,
  path: "$id",
  component: CourseDetailPage,
});
```

```
// Inside the component — fully typed via Route
function CourseDetailPage() {
  const { id } = Route.useParams();
  //      ^? string — TypeScript knows this, no "from" needed

  const { data: course } = useGetCoursesId(id);
  return <div>{course?.name}</div>;
}
```

Search Parameters

Query strings like `/courses?semester=spring&minCredits=4`.

TanStack Router makes search params **validated and typed** — not raw strings.

```
import { z } from "zod";

const coursesRoute = createRoute({
  getParentRoute: () => rootRoute,
  path: "/courses",
  // Validate search params with Zod
  validateSearch: z.object({
    semester: z.enum(["spring", "fall"]).optional(),
    minCredits: z.number().min(1).optional(),
  }),
  component: CoursesPage,
});
```

Using Search Parameters

```
function CoursesPage() {
  const { semester, minCredits } = Route.useSearch();
  //      ^? "spring" | "fall" | undefined — no "from" needed

  const navigate = Route.useNavigate();

  const setSemester = (s: "spring" | "fall") => {
    navigate({
      search: (prev) => ({ ...prev, semester: s }),
    });
  };

  return (
    <div>
      <SemesterFilter value={semester} onChange={setSemester} />
      <CourseList semester={semester} minCredits={minCredits} />
    </div>
  );
}
```

Search params are part of the URL — shareable, bookmarkable, back-button friendly.

Real-World: E-shop Filters

Imagine an e-shop. User selects category, price range, sorts by rating, goes to page 3:

```
https://shop.com/products?category=shoes&minPrice=500&sort=rating&page=3
```

Now they **send the link to a friend**. Friend opens it → sees the exact same view.

Now try that with `useState`. The friend opens the link → sees the default page. All filter state is lost.

This is why search params matter. Any UI state that should be shareable or survive a refresh belongs in the URL — filters, pagination, sorting, selected tabs, search queries...

Search Params vs useState

```
// ❌ State lost on refresh, not shareable
const [semester, setSemester] = useState<string>();

// ✅ URL is the source of truth
// /courses?semester=spring — survives refresh, sharable
const { semester } = Route.useSearch();
```

Search params survive refresh, are shareable via URL, work with the back button, and are type-safe with Zod validation. `useState` gives you none of that.

Rule of thumb: If a user would want to bookmark or share the state — it belongs in the URL.

Task 2 — Detail Page & Search Params (~15 min)

1. **Course detail route** — create `routes/courses/$id.tsx`
   - Use `Route.useParams()` to get the course ID
   - Display course details using the generated React Query hook
   - Add `<Link>` from the course list to each course's detail page
2. **Semester filter via search params** — update `routes/courses/index.tsx`
   - Add `validateSearch` with a Zod schema: `semester` (optional enum)
   - Use `Route.useSearch()` to read the current filter
   - Use `Route.useNavigate()` to update the search param on filter change
   - Replace any `useState` for semester with search params

Verify: `/courses?semester=spring` shows filtered results. Refresh → filter persists. Back button works.

Devtools

```
// src/routes/__root.tsx
import { TanStackRouterDevtools } from "@tanstack/react-router-devtools";

function RootLayout() {
  return (
    <>
      <nav>...</nav>
      <main><Outlet /></main>

      {/* Shows route tree, params, search state in dev */}
      <TanStackRouterDevtools position="bottom-right" />
    </>
  );
}
```

Devtools show the full route tree, current match, params, search params — invaluable for debugging.

Building the Frontend App

Layouts, guards, and architecture patterns

Nested Layouts

Layouts let you share UI across groups of routes — without re-rendering on navigation.

```
__root.tsx                  → Navbar + <Outlet>
├── students/
│   ├── route.tsx           → Students layout + <Outlet>
│   ├── index.tsx           → Student list
│   └── $id.tsx             → Student detail
├── courses/
│   ├── route.tsx           → Courses layout + <Outlet>
│   ├── index.tsx           → Course list
│   └── $id.tsx             → Course detail
```

When navigating from `/students` to `/students/123`:

- `__root.tsx` — stays mounted (navbar doesn't re-render)
- `students/route.tsx` — stays mounted (header doesn't re-render)
- Only the `<Outlet>` content swaps (index → $id)

Layout Routes — Example

```tsx
// src/routes/courses/route.tsx — layout for all /courses/* routes
import { createFileRoute, Outlet } from "@tanstack/react-router";

export const Route = createFileRoute("/courses")({
  component: CoursesLayout,
});

function CoursesLayout() {
  return (
    <div>
      <h1 className="text-2xl font-bold mb-4">Courses</h1>
      <Outlet /> {/* /courses/ or /courses/$id renders here */}
    </div>
  );
}
```

```tsx
// src/routes/courses/index.tsx — the actual /courses list page
export const Route = createFileRoute("/courses/")({
  component: CoursesListPage,
});
```

Task 3 — Add a Layout Route (~10 min)

1. **Create `routes/courses/route.tsx`** — a layout for all `/courses/*` routes
   - Add a heading ("Courses") and any shared UI
   - Render `<Outlet />` for child content
2. **Do the same for `routes/students/`** — create `students/route.tsx`
   - The layout should have a heading ("Students")
   - `students/index.tsx` already exists — it becomes the child

Verify: Navigate between `/courses` and `/courses/123` — the heading stays, only the content below changes. Check the devtools route tree.

Core Concepts — Checkpoint

This is what you **need to know** to build a routed app:

- **URL anatomy** — path, params, search params
- **Route tree** — file-based routing, `route.tsx` for layouts
- **`Link`** — type-safe navigation with params and search
- **Path params** (`$id`) — for resource identity
- **Search params** — for filters, pagination, sorting (validated with Zod)
- **Nested layouts** — shared UI (navbar, headers) via `route.tsx` + `<Outlet />`

Everything that follows builds on top of this foundation.

Going Further

Patterns you'll use as your app grows

Pathless Layout Routes

Sometimes you want a shared layout for routes that **don't share a URL prefix**.

The `_` prefix on a folder makes it a **pathless layout** — it wraps children without adding a URL segment:

```
routes/
├── _authenticated/                    ← _ prefix = no URL segment
│   ├── route.tsx                  → Layout (auth check + <Outlet />)
│   ├── students.tsx               → /students  (not /_authenticated/students)
│   └── courses.tsx                → /courses
├── login.tsx                      → /login (no layout wrapper)
```

Compare with a **normal folder** like `courses/`:

- `courses/` → adds `/courses` to the URL
- `_authenticated/` → adds **nothing** to the URL, just wraps children in a layout

Route Guards — Protecting Pages

Use the `beforeLoad` hook to check auth before rendering:

```
// src/routes/_authenticated/route.tsx
export const Route = createFileRoute("/_authenticated")({
  beforeLoad: async ({ location }) => {
    const isAuthenticated = checkAuth();

    if (!isAuthenticated) {
      throw redirect({
        to: "/login",
        search: { redirect: location.href },
      });
    }
  },
  component: () => <Outlet />,
});
```

Runs before the route renders. All child routes inherit the guard.

Data Loading with Loaders

Load data **before** the component renders — no loading spinner on every navigation.

```
// src/routes/courses/$id.tsx
export const Route = createFileRoute("/courses/$id")({
  loader: async ({ params }) => {
    const course = await fetchCourse(params.id);
    return { course };
  },
  component: CourseDetailPage,
});

function CourseDetailPage() {
  const { course } = Route.useLoaderData();
  //      ^? fully typed — same shape as loader return

  return <h1>{course.name}</h1>;
}
```

Why Loaders + React Query?

Loaders alone fetch data — but **React Query** adds a caching and synchronization layer on top.

- **Loader** = "fetch this data before the route renders"
- **React Query** = "cache it, refetch in background, share across components"

Together:

- **First visit** — loader prefetches into React Query cache → instant render
- **Return visit** — cache hit, no network request at all
- **Background refetch** — stale data is shown immediately, fresh data replaces it
- **Mutations** — invalidate cache after create/update/delete → list auto-refreshes

Without React Query, every navigation re-fetches. With it, data is smart and shared.

Loaders + React Query — In Practice

In our project, we use React Query for data fetching. Combine them:

```
import { queryOptions } from "@tanstack/react-query";

const courseQueryOptions = (id: string) =>
  queryOptions({
    queryKey: ["courses", id],
    queryFn: () => fetchCourse(id),
  });

export const Route = createFileRoute("/courses/$id")({
  loader: ({ context: { queryClient }, params }) => {
    // Ensures data is in cache when component renders
    return queryClient.ensureQueryData(courseQueryOptions(params.id));
  },
  component: CourseDetailPage,
});

function CourseDetailPage() {
  const { id } = Route.useParams();
  const { data } = useSuspenseQuery(courseQueryOptions(id));
  return <h1>{data.name}</h1>;
}
```

React Suspense & Error Boundaries

Two React primitives for handling **async states declaratively** — at the parent level.

- `<Suspense>` — catches loading states (thrown Promises). Shows a fallback until children are ready.
- `<ErrorBoundary>` — catches thrown errors. Shows a fallback instead of crashing the whole app.

```
<ErrorBoundary fallback={<p>Something went wrong.</p>}>
  <Suspense fallback={<p>Loading…</p>}>
    <CoursesList />
  </Suspense>
</ErrorBoundary>
```

The parent controls both loading and error UI. The child component doesn't need any `if (isPending)` / `if (isError)` checks.

Why Suspense? — Before & After

```
// Without Suspense — every component handles its own loading & errors
function CoursesList() {
  const { data, isPending, isError } = useGetCourses();
  if (isPending) return <p>Loading…</p>; // ← boilerplate in every component
  if (isError) return <p>Error!</p>;     // ← boilerplate in every component
  return <ul>{data.map(c => <li key={c.id}>{c.name}</li>)}</ul>;
}
```

```
// With Suspense — parent handles loading & errors, component just renders
<ErrorBoundary fallback={<p>Error!</p>}>
  <Suspense fallback={<p>Loading…</p>}>
    <CoursesList />
  </Suspense>
</ErrorBoundary>

function CoursesList() {
  const { data } = useSuspenseQuery(...); // ← data is always defined!
  return <ul>{data.map(c => <li key={c.id}>{c.name}</li>)}</ul>;
}
```

useQuery vs useSuspenseQuery

	`useQuery`	`useSuspenseQuery`
Loading state	You handle it (`isPending`)	Suspense boundary handles it
Error state	You handle it (`isError`)	ErrorBoundary handles it
`data` type	`TData | undefined`	`TData` (always defined)
Works without Suspense	✅	❌ needs `<Suspense>` parent

Rule of thumb: If data is required to render (e.g. course detail) → `useSuspenseQuery`.
Optional/background data (e.g. notification count) → `useQuery`.

useQuery vs useSuspenseQuery — In Code

```
// useQuery — data can be undefined, you must check
const { data, isPending, isError } = useQuery(opts);
//      ^? Course | undefined

// useSuspenseQuery — data is guaranteed, component is simpler
const { data } = useSuspenseQuery(opts);
//      ^? Course — never undefined
```

Suspense in TanStack Router

You don't need to write `<Suspense>` or `<ErrorBoundary>` yourself — TanStack Router wires them up per route:

```
export const Route = createFileRoute("/courses/$id")({
  loader: ...,
  pendingComponent: () => <Spinner />,   // = Suspense fallback
  errorComponent: ({ error }) => ...,    // = ErrorBoundary fallback
  component: CourseDetailPage,
});
```

You can also set **defaults for all routes** in the router config:

```
const router = createRouter({
  routeTree,
  defaultPendingComponent: () => <Spinner />,
  defaultErrorComponent: ({ error }) => <p>Error: {error.message}</p>,
});
```

Per-route settings override the defaults.

Prefetching

TanStack Router can **prefetch** data when the user hovers over a link.

```
// preload="intent" — prefetch on hover/focus (default)
<Link to="/courses/$id" params={{ id: course.id }} preload="intent">
  {course.name}
</Link>
```

Combined with loaders + React Query: by the time the user clicks, the data is already cached. **Zero-latency navigation.**

Task 4 — Loader + React Query (~10 min)

1. **Add a loader to `courses/$id.tsx`**
   - Use the query options generated by Kubb (`getCoursesByIdQueryOptions`)
   - Call `queryClient.ensureQueryData(...)` in the loader with those options
   - In the component, use `useSuspenseQuery(...)` with the same query options
2. **Add error handling**
   - Add `pendingComponent` — show a loading message
   - Add `errorComponent` — show an error with a back link
   - Add `notFoundComponent` — show "Course not found"
3. **Test prefetching** — hover over a course link, check the Network tab
   - Data should be fetched on hover, navigation should feel instant

Verify: Navigate to a course detail → no loading spinner (data prefetched). Try an invalid ID → error UI shows.

How Navigation Works — Mental Model

What happens when a user clicks a `<Link>`?

```
1. Navigate              → URL changes to /courses/123
2. Match route           → Router finds the matching route: /courses/$id
3. Run guard             → beforeLoad checks access (may redirect)
4. Load data             → loader prefetches data into React Query cache
5. Show pending UI       → pendingComponent is shown while loading
6. Render page           → component renders with params + ready data
7. Reuse cache later     → next visit may be instant, without refetch
```

What we've learned so far about structuring a React app:

Routing & URLs

- Every page has its own URL — use file-based routing
- UI state that should be shareable (filters, pagination) → search params, not `useState`
- Fetch data in loaders, not in `useEffect` — no spinners on navigation

Components & code organization

- Route-specific components live with their route (`-components/`)
- Shared UI components (Button, Card) live in top-level `components/`
- Generated API hooks (Kubb) — don't write fetch code by hand

Layouts & structure

- Shared UI (navbar, headers) → layout `route.tsx` + `<Outlet />`
- Group routes by auth/access → pathless `_` layouts with guards

The `-` Prefix Convention

Files and folders prefixed with `-` are **ignored by the router** — they don't become routes.

Use them for route-specific components, utils, or hooks that live next to their route:

```
routes/courses/
├── route.tsx              ← /courses layout
├── index.tsx              ← /courses page
├── $id.tsx                ← /courses/$id page
├── -components/           ← ignored by router!
│   ├── CourseCard.tsx
│   └── SemesterFilter.tsx
├── -hooks/
│   └── useCourseFilters.ts
└── -utils/
    └── formatCredits.ts
```

Anything prefixed with `-` is **not** a route — it lives close to where it's used, but the router ignores it.

Architecture Overview

```
src/
├── routes/
│   ├── __root.tsx                    ← Shell: navbar, footer
│   ├── _authenticated/
│   │   ├── route.tsx                 ← Auth guard layout
│   │   ├── students/
│   │   │   ├── index.tsx             ← /students
│   │   │   ├── $id.tsx               ← /students/$id
│   │   │   └── -components/          ← student-specific UI
│   │   └── courses/
│   │       ├── index.tsx             ← /courses
│   │       ├── $id.tsx               ← /courses/$id
│   │       └── -components/          ← course-specific UI
│   └── login.tsx
├── components/                       ← Shared UI (Button, Card...)
├── generated/                        ← API hooks (Kubb)
└── lib/                              ← Utilities
```

`-components/` = route-specific, lives with the route. **`components/`** = shared across the app.

TanStack Start

The full-stack framework built on TanStack Router

Router vs Start

TanStack Router = routing library (what we've covered so far)

- Route trees, params, search params, loaders, layouts, code splitting
- File-based routing via `@tanstack/router-plugin`
- Client-side only — runs entirely in the browser

TanStack Start = full-stack meta-framework (like Next.js for React)

- Everything Router has, **plus:**
- Server-Side Rendering (SSR), Static Site Generation (SSG)
- **Server functions** — type-safe RPCs between client and server
- Middleware, server routes, deployment presets
- SPA mode — opt out of SSR entirely

Server Functions — Start's Killer Feature

Define backend logic inline — type-safe across the boundary:

```
import { createServerFn } from "@tanstack/react-start";

const getSecretData = createServerFn({ method: "GET" })
  .handler(async () => {
    // This code runs ONLY on the server
    const secret = process.env.SECRET_API_KEY;
    const data = await fetch(`https://api.example.com?key=${secret}`);
    return data.json();
  });

// Call it from the client — looks like a normal async function
const data = await getSecretData();
```

No separate API layer needed. Types flow from server to client automatically.

Rendering Modes

TanStack Start supports multiple rendering strategies:

```
SPA mode          → Client-only, static hosting (CDN)
                    Our current approach — simplest

SSR               → Server renders HTML on each request
                    Better SEO, faster first paint

SSG               → HTML generated at build time
                    Best for content that rarely changes

Selective SSR     → Per-route choice of SSR vs client-only
                    Most flexible — SSR where it matters
```

We use **SPA mode** — simpler, cheaper hosting, no hydration complexity.

But knowing SSR exists helps you choose the right tool for future projects.

Checkpoint: Key Takeaways

- **URLs are UI** — filters, selections, page state belong in the URL
- **File-based routing** is part of **TanStack Router** (via Vite plugin)
- **Type safety everywhere** — params, search params, loader data
- **Layouts nest** — shared UI without re-renders, pathless layouts for groups
- **Guards with `beforeLoad`** — auth checks before rendering
- **Loaders + React Query** — data ready before render, prefetch on hover
- **TanStack Start** adds SSR, SSG, server functions on top of Router

Questions?

Summary

What We Covered

1. **Why routing** — URLs, navigation, browser history, shareability
2. **TanStack Router** — route trees, file-based routing, `Link`, type-safe params and search params
3. **Layouts** — nested layouts, pathless layout routes, `<Outlet />`
4. **Route guards** — `beforeLoad` + `redirect` for auth
5. **Data loading** — loaders, React Query integration, prefetching
6. **Pending UI & errors** — progress indicators, error/not-found boundaries
7. **TanStack Start** — SSR, SSG, server functions — the full-stack layer

Useful Links

- TanStack Router Docs
- TanStack Router — File-Based Routing
- TanStack Router — Search Params
- TanStack Router — Data Loading
- TanStack Start Docs

Next Seminar Preview

Seminar 07 — Interactive forms & state management — React Hook Form, watch, control, shadcn integration